INDUSTRIAL STRENGTH SOFTWARE SERVICES
TECHNICAL REPORT · ISSS-TR-0502

GROUP COMMIT FOR SYNC_ON_PUT

SHARING FSYNC ACROSS CONCURRENT WRITERS IN YABIIR

AUTHOR
Javier Fernández

CONTACT
jfernand@me.com

ISSUED
2026-09-17

ABSTRACT

`yabiir` is a from-scratch Rust implementation of Bitcask, the log-structured hash-table key/value store described in Sheehy & Smith's 2010 paper. With `Options::sync_on_put` enabled — the opt-in that makes every write durable against a true power outage, not just a process crash — every `put` / `delete` call previously issued its own `fsync`, even though those calls were already fully serialized through the same lock. Concurrent callers were paying for N separate fsyncs when the disk only needed to be told "sync" once to cover all of them. This report explains the problem, the group-commit coordinator that fixes it, why it doesn't weaken the guarantee `sync_on_put` exists to provide, and the controlled A/B used to verify the fix.

DOCUMENT CONTROL

REPOSITORY	yabiir — Rust, Bitcask log-structured storage engine
COMPONENT	`src/commit.rs` (new), `src/engine.rs`, `src/datafile.rs`, `src/merge.rs`
TRIGGER	Investigating sync_on_put's real cost via `src/bin/loadtest.rs` — see § 1
VERIFICATION	Controlled A/B via a `git worktree` at the prior commit, real `loadtest` runs — see § 4

01 THE PROBLEM

`Options::sync_on_put` trades throughput for a stronger durability guarantee: without it, every write is `flush`ed (a `write` syscall — visible to concurrent readers, survives a process crash) but not `fsync`'d, so the last few writes before a true power outage can be lost. With it, `Engine::append` called `ActiveFile::sync` (flush + `fsync_data`) on every single `put` / `delete`:

SRC/ENGINE.RS — ENGINE::APPEND (BEFORE THIS CHANGE)

```rust
fn append(&self, encoded: &format::EncodedEntry) -> Result<(u32, u64)> {
    let active_lock = self.active.as_ref().ok_or(Error::ReadOnly)?;
    let mut active = active_lock.lock().unwrap();
    let (file_id, value_pos, _total_len) = active.append(encoded)?;
    if self.opts.sync_on_put {
        active.sync()?;
    }
    if active.len() >= self.opts.max_file_size {
        active.sync()?;
        let new_id = self.next_file_id.fetch_add(1, Ordering::SeqCst);
        *active = ActiveFile::create(&self.dir, new_id)?;
    }
    Ok((file_id, value_pos))
}
```

A real `loadtest` run made the cost concrete: with `sync_on_put`, `put`'s median latency jumped from roughly 2.3 µs (unsynced) to 24.9 ms — over 11,000× — and, measured again in a clean baseline for this report (8 concurrent writer threads, no merge running, so the number reflects fsync contention alone), the tail was far worse than the median suggested:

PUT P50	49.24 MS
`put` p90	136.95 ms
`put` p99	270.16 ms
`put` max	525.15 ms
`put` throughput	≈97/s

That p50-to-max spread — 49 ms typical, 525 ms occasionally, with 8 threads issuing writes at a modest keyspace — is the signature of **unbatched** fsync contention: `Engine::append` already serializes every write through the same `Mutex<ActiveFile>`, so eight threads' fsyncs run strictly one after another regardless of how close together they arrive. Two threads that both wanted to write within the same millisecond still pay for two full fsyncs, back to back, purely because neither knows the other is waiting.

> **WHY THIS IS EXACTLY A GROUP-COMMIT OPPORTUNITY**
> This is the same problem write-ahead-log databases have solved for decades under the name **group commit**: `fsync`'s cost comes from the physical disk operation, not from how many logical writes it happens to cover — an fsync that flushes 1 buffered write and an fsync that flushes 10 buffered writes cost roughly the same. If several callers are already waiting on the same lock when one of them is about to fsync, there's no reason for the other N−1 to each pay for their own.

Crucially, this is orthogonal to the two earlier fixes in this investigation (dropping the active-file read lock, and the merge flush-batching change in `docs/merge-batched-flushing.typ`) — neither of those touches `sync_on_put`'s fsync path at all. `sync_on_put`'s cost is entirely `Engine::append`'s own, and it was entirely unbatched.

02 DESIGN

The core idea: several concurrent writers can share one fsync, but each caller must still **block until its own data's** fsync has completed before `put` / `delete` returns — sharing a fsync must never let one caller

return `Ok` before its bytes are actually durable. That's a stronger requirement than merge's batching (§ 2 of the companion report): merge could defer a keydir repoint arbitrarily, because nothing outside merge was waiting on it. Here, the caller **is** waiting, synchronously, for the return value.

2.1 A GENERATION-COUNTER COORDINATOR

`src/commit.rs` is a new, small module — `GroupCommit` — that doesn't know anything about `ActiveFile` or `Engine`. It tracks two monotonic counters under one `Mutex` / `Condvar`: `pending` (how many writes have been recorded so far) and `durable` (the highest one known to be covered by a completed fsync), plus a `syncing` flag so only one thread performs a fsync at a time.

SRC/COMMIT.RS — THE COORDINATION LOOP

```rust
pub(crate) fn commit(
    &self,
    target_gen: u64,
    do_fsync: impl Fn() -> io::Result<()>,
) -> io::Result<()> {
    let mut state = self.state.lock().unwrap();
    loop {
        if state.durable >= target_gen {
            return Ok(());
        }
        if !state.syncing {
            state.syncing = true;
            let covers_up_to = state.pending;
            drop(state);
            let result = do_fsync();
            state = self.state.lock().unwrap();
            state.syncing = false;
            if result.is_ok() {
                state.durable = state.durable.max(covers_up_to);
            }
            self.cv.notify_all();
            result?;
        } else {
            state = self.cv.wait(state).unwrap();
        }
    }
}
```

The first caller to reach `commit` while no fsync is in flight becomes the **leader**: it runs `do_fsync` with no lock held (so appends and other threads' bookkeeping aren't blocked while the fsync itself is slow), then marks every generation up to whatever was `pending` **when it started** as durable and wakes everyone. Every other thread that arrives while `syncing` is already true just waits — it doesn't need a fsync of its own, because the one already in flight started **after** its own write completed (see the ordering argument in § 2.3), so it's guaranteed to cover it.

`record_pending` is called separately, under the **append** lock, at the moment a write is appended:

SRC/COMMIT.RS — RECORDING A WRITE'S GENERATION

```rust
pub(crate) fn record_pending(&self) -> u64 {
    let mut state = self.state.lock().unwrap();
    state.pending += 1;
    state.pending
}
```

2.2 `ENGINE::APPEND`: APPEND UNDER THE LOCK, COMMIT OUTSIDE IT

```rust
fn append(&self, encoded: &format::EncodedEntry) -> Result<(u32, u64)> {
    let active_lock = self.active.as_ref().ok_or(Error::ReadOnly)?;
    let (file_id, value_pos, target_gen) = {
        let mut active = active_lock.lock().unwrap();
        let (file_id, value_pos, _total_len) = active.append(encoded)?;
        let target_gen = self.group_commit.record_pending();
        if active.len() >= self.opts.max_file_size {
            active.sync()?;
```

```
            self.group_commit.mark_all_durable();
            let new_id = self.next_file_id.fetch_add(1, Ordering::SeqCst);
            *active = ActiveFile::create(&self.dir, new_id)?;
        }
        (file_id, value_pos, target_gen)
    };

    if self.opts.sync_on_put {
        self.group_commit.commit(target_gen, || {
            let mut active = active_lock.lock().unwrap();
            let fd = active.sync_handle()?;
            drop(active); // don't hold the append lock across the fsync itself
            fd.sync_data()
        })?;
    }
    Ok((file_id, value_pos))
}
```

Two details make this correct:

1. **`record_pending` runs under the same lock as the append it belongs to.** That's what guarantees generation numbers are assigned in real append order — if thread A's `write_all` happens-before thread B's (because they share one `Mutex<ActiveFile>`), A's generation number is lower than B's too. A fsync that covers generation **N** is therefore guaranteed to cover every write with a lower generation number, because all of them were already physically written (via `append`'s own internal flush) before their generation number was even handed out.
2. **The leader's fsync always targets the current active file, fetched fresh.** `do_fsync` doesn't capture a file handle up front — it locks `active` again, right when it actually runs, clones a fresh descriptor via `ActiveFile::sync_handle` (a `flush` plus `try_clone`, new in this change — see § 2.4), and only **then** releases the lock before calling `sync_data` on the clone. This is what lets the fsync happen without holding `active` for its whole duration, while still always fsyncing whatever file is genuinely current at that moment — correct even if a rotation happened between this write's `append` and its `commit` call (see § 2.5).

2.3 `ACTIVEFILE::SYNC_HANDLE`: FSYNC WITHOUT HOLDING THE WRITE LOCK

```
pub(crate) fn sync_handle(&mut self) -> io::Result<File> {
    self.writer.flush()?;
    self.writer.get_ref().try_clone()
}
```

`fsync` operates at the OS/inode level, not per file descriptor — a `sync_data()` call on a cloned descriptor durably persists exactly the same bytes a call on the original would. This lets the group-commit leader do the (potentially tens-of-milliseconds) fsync syscall **without** blocking every other thread trying to append a new entry in the meantime — holding `Mutex<ActiveFile>` across a slow disk I/O call is exactly the anti-pattern this codebase has been careful to avoid elsewhere (`DataFileSet::read_at` follows the same "clone/fetch under the lock, do the I/O outside it" shape).

2.4 ROTATION ALREADY FSYNCS — TELL THE COORDINATOR, DON'T FSYNC TWICE

Rotation (crossing `max_file_size`) calls `ActiveFile::sync` unconditionally, regardless of `sync_on_put` — that's existing, unrelated behavior, kept as is. If a write's own append happened to trigger (or merely precede) a rotation, that write is **already** durable by the time it would otherwise enter the group-commit coordinator — rotation's fsync covers everything appended to the outgoing file, and it runs synchronously, still under the same append lock, before any thread reaches `commit`. `mark_all_durable` records that directly:

```
if active.len() >= self.opts.max_file_size {
    active.sync()?;
    self.group_commit.mark_all_durable();
    let new_id = self.next_file_id.fetch_add(1, Ordering::SeqCst);
    *active = ActiveFile::create(&self.dir, new_id)?;
}
```

Without this, a writer whose entry landed right before a rotation would still work correctly (it would just wait for the **next** round's fsync, on the **new** file, which is unrelated to its own data but eventually happens anyway since `sync_on_put` keeps producing rounds) — but it's a needless wait for a fsync that already happened. `merge`'s own force-rotation step (see `src/merge.rs`'s module doc on file-ordering correctness) gets the same treatment, for the same reason.

> **WHAT HAPPENS IF THE ACTIVE FILE ROTATES AGAIN MID-WAIT**
> Suppose thread T appends to file A, some other thread's append later triggers file A's rotation to file B (T didn't cause it, but it still durably covers T's entry via rotation's unconditional fsync and the `mark_all_durable` call above), and **then** T reaches `commit`. Since `durable` was already advanced past T's generation before T ever calls `commit`, the `if state.durable >= target_gen { return Ok(()) }` check at the top of the loop fires immediately — T never becomes leader, never fsyncs file B, and returns as soon as it observes the state rotation already set. No redundant fsync, no wrong-file fsync, no wait.

03 CRASH SAFETY

> **THE SYNC_ON_PUT GUARANTEE IS UNCHANGED: STRONGER AMORTIZED COST, SAME PROMISE PER CALL**
> `put` / `delete` still only return `Ok` once the caller's own write is durable — group commit changes **how many fsync syscalls** that requires in aggregate under concurrency, never **whether** a given call's data is fsynced before that call returns. The generation-counter ordering argument in § 2.3 is precisely what makes this true: a caller only ever observes `durable >= target_gen` after a fsync that genuinely ran after its own `write_all` completed.

A failed fsync is handled simply, and deliberately not with more sophistication than the problem calls for: the leader's own `do_fsync` error propagates directly to that leader's `put` / `delete` caller (an `Err` — correctly telling that caller its durability is **not** confirmed), `durable` is left unchanged, and `syncing` is reset so the next waiting thread retries as a new leader. Under a persistent disk error this retries indefinitely rather than giving up — accepted as a reasonable default (there's no universally right answer to "what should a database do when fsync keeps failing"; PostgreSQL's own well-known "fsyncgate" discussion is the classic reference here), and out of scope to harden further for this pass.

04 VERIFICATION

4.1 CORRECTNESS

- The full test suite (59 library tests, up from 53 before this change, plus `tests/model.rs`'s property-based model test) passes.
- `src/commit.rs` has its own focused unit tests for the coordinator in isolation: a single writer sees exactly one fsync; `mark_all_durable` satisfies a waiting `commit` with **zero** fsyncs; a deterministic (not timing-based) test blocks a "leader" mid-fsync via a channel, spawns 7 followers, confirms they've all reached `commit` and are waiting, then releases the leader — and asserts exactly 1 fsync ran for all 8; a failed fsync propagates to its caller and a subsequent attempt for the same generation retries and can succeed.
- `engine::tests::sync_on_put_round_trips_correctly` and `engine::tests::concurrent_sync_on_put_writers_share_fsyncs` exercise the same property through the real `Engine`, the latter asserting (via a test-only fsync-call counter) that 32 concurrent writers share well under 32 real fsyncs, then verifying every write is actually readable back.
- `tests/model.rs`'s proptest run at 1,000 cases (5× the default) with no failures.

4.2 PERFORMANCE — A CONTROLLED A/B

Following the same methodology as the merge flush-batching report: a `git worktree` checked out the commit immediately before this change, built in release mode there, and ran the identical `loadtest` invocation (8 threads, 20,000 keys, `--sync-on-put`, no merge) against both binaries — isolating this one change from everything else already landed (the active-file read-lock removal, the `ahash` hasher swap, and merge's flush-batching, all separate fixes from the same profiling effort).

METRIC	BEFORE	AFTER
`put` p50	49.24 ms	12.82 ms
`put` p90	136.95 ms	13.68 ms
`put` p99	270.16 ms	16.73 ms
`put` max	525.15 ms	22.88 ms
`put` throughput	≈97/s	≈410/s
`delete` p50	6.56 ms	12.71 ms
`delete` max	443.93 ms	20.89 ms
combined write throughput	≈146/s	≈609/s

Two things stand out beyond the raw throughput gain (roughly 4.2× combined):

- **The tail collapsed, not just the median.** Before this change, `put`'s max latency (525 ms) was over 10× its own p50 (49 ms) — the signature of unlucky writers occasionally queueing behind several others' independent fsyncs. After, p50/p90/p99/max are all within a tight 12.8–22.9 ms band. For a design goal centered on **predictable** latency (see the project README), this tail compression matters at least as much as the average speedup.
- **`delete`'s p50 went up slightly (6.56 ms → 12.71 ms) even though its own tail also collapsed (max 443.93 ms → 20.89 ms).** This isn't a regression: in the "before" run, `delete`'s lucky low p50 reflects deletes that happened to run when no other thread's fsync was in flight ahead of them — a favorable-scheduling artifact of an unbatched, highly variable system, not a stable baseline. Under group commit, every write (`put` or `delete`) converges to roughly the same cost — one shared fsync's worth, amortized — which is exactly the intended effect.

05 WHAT THIS DOESN'T ADDRESS

- **The always-on per-write `flush` cost (independent of `sync_on_put`).** Every `put`/`delete` still calls `ActiveFile::append`, which flushes unconditionally for read-your-own-write visibility — measured earlier in this investigation at ≈9.4% of `put`'s own sampled CPU time. That's a cheap `write` syscall, not a `fsync`, and batching it the way merge batches its own flush (`docs/merge-batched-flushing.typ`) isn't safe for `put`/`delete` in general: an external reader could legitimately expect to see a key immediately after `put` returns, which is precisely the guarantee that flush exists to uphold.
- **fsync failure handling stays simple (§ 3)** — retry-as-new-leader indefinitely, no backoff, no giving up. Fine for this codebase's current scope; a production deployment on unreliable storage might want more.
- **The known merge/recovery file-ordering gap** documented in `src/merge.rs`'s module comment is unrelated to this change and remains exactly as it was.

06 ADDENDUM, 2026-11-12: WHAT CHANGED SINCE

This report describes the change as it was made on 2026-09-17, and the code it quotes is the code of that day. Three things are different now.

A failed fsync is no longer retried. § 3 called retrying "a reasonable default". It isn't safe on Linux: the kernel reports a failed write-back to the next `fsync` and then marks the pages clean, so a second `fsync` of the same file can succeed although the data never reached the disk. A waiter that retried as the new leader could therefore return `Ok` for a write that was lost. Now a failed fsync fails every write that wasn't durable when the fsync started, the leader's and the waiters' alike, and they stay failed whatever syncs succeed meanwhile or afterwards; the writer drops the file before anyone else can append to it, and the next write starts a new one. A caller that gets an error can't tell whether its write survived. A caller that gets `Ok` can. The coordinator also recovers if the leader's fsync panics, instead of leaving every later writer waiting.

The code moved. The coordinator is still `src/commit.rs` (now `crates/yabiir/src/commit.rs`), with the counters renamed `newest_generation` and `durable_generation`. `Engine::append` became `Writer::append` in `src/writer.rs`, which owns the active file, the file-id counter and the coordinator behind one lock. A write that doesn't ask for `sync_on_put` no longer takes a generation at all.

The keydir is updated inside the append lock, before the shared fsync, not after it. That closed a race in which two writes to one key could update the keydir out of log order. Its cost is stated in the README: with `sync_on_put`, another thread can read a value just before it is durable. The writer itself still returns only after the fsync.

The merge/recovery ordering gap that the last section calls unrelated is closed too; `src/merge/mod.rs` explains how.